

Data Science project: Predicting the price of real estate properties

Code ▾

Jakhongir Shirinov

#

Predicting Price Based on Features.

About Dataset (Context)

The Data set contains following columns for real estates in Tashkent, Uzbekistan:

- Tashkent_city_Districts - the district the real-estate located in
- Price_in_USD - price of the estate in USD
- Area - the distance of the property (how far/close) to downtown
- Size - total size of the unit in square meters
- Rooms - number of rooms
- Type - new or old
- Group - Grouped by the numbers which represent the district in which the property is located.
- Date - the date when the property was built
- Latitude and Longitude - geographical coordinates

The objective of the coursework is to build a Regression Model to predict the price of "Real estate" properties based on the available features, such as "Area," "Rooms," "Type," "City_District," "Date", "Latitude", "Longitude", "Type" and "Group." This involves - linear regression, - K-NN, - Random Forest and - Gradient Boosting Machine (GBM).

The unit of observation: Individual properties. Each row in my data set represents a specific property with various characteristics.

The population I hope my data sample represents is likely the real estate market in Tashkent City. My data set captures information about different properties within the city, providing a snapshot of their prices, characteristics, and locations. The goal is to use this sample to build models to understand and predict real estate prices based on the given features.

Hide

```
library(data.table)
library(httr)
library(jsonlite)
library(rstudioapi)
library(XML)
library(stringr)
library(ggcorrplot)
library(ggExtra)
library(ggplot2)
library(ggribes)
library(ggsci)
library(ggthemes)
library(RColorBrewer)
library(svglite)
library(viridis)
library(MASS)
library(mnormt)
library(plot3D)
library(viridisLite)
library(FNN)
library(nnet)
library(rpart)
library(rpart.plot)
library(viridisLite)
library(caret)
library(pROC)
library(imputeTS)
library(forecast)
library(dplyr)
library(class)
library(GGally)
library(car)
```

Collection and preparation of the Data Sets.

For the preparation of the data set, first of all, I set R's working directory and imported file from the local memory. To do so I used "fread" of the "data.table" library.

Further steps are done as well: - Combined two data frames by column binding. - Removed the words "Tashkent," and "district" from the lines of a Tashkent_city_districts column in my data set, to do that I used the 'gsub' function in R. - Reordered the columns - Changed the format of the variables

[Hide](#)

```
setwd("c:/Users/jakho/OneDrive/MLU/DATA SCIENCE/Project 1")

Real_estate <- fread("c:/Users/jakho/OneDrive/MLU/DATA SCIENCE/Project 1/RE Dataset in1.csv")

Real_estate[, "Date" := as.Date(Date, format = "%m/%d/%Y")]
Real_estate <- na.omit(Real_estate)
Real_estate[, Source.Name := NULL]

Size_and_loc_in<- fread("c:/Users/jakho/OneDrive/MLU/DATA SCIENCE/Project 1/Size and location in1.csv")

Real_estate <- cbind(Real_estate, Size_and_loc_in)
```

```
Warning in as.data.table.list(x, keep.rownames = keep.rownames, check.names = check.names, :
Item 1 has 458 rows but longest item has 459; recycled with remainder.
```

[Hide](#)

```

setnames(Real_estate, "price", "Price_in_USD")
setnames(Real_estate, "rooms", "Rooms")
setnames(Real_estate, "type", "Type")
setnames(Real_estate, "City_District", "Tashkent_city_districts")

Real_estate[, "Group" := .GRP, by = "Tashkent_city_districts"]
Real_estate[, Group := as.factor(Group)]

Real_estate$Tashkent_city_districts <- gsub("Tashkent,", "", Real_estate$Tashkent_city_districts, ignore.case = TRUE)
Real_estate$Tashkent_city_districts <- gsub("district", "", Real_estate$Tashkent_city_districts, ignore.case = TRUE)

new_column_order <- c("Tashkent_city_districts", "Price_in_USD", "Area", "Size", "Rooms", "Type", "Latitude", "Longitude", "Group", "Date")
setcolorder(Real_estate, new_column_order)

#"Date" variable appropriately to convert it into a numerical format that the neural network can understand and process effectively.
Real_estate$Year <- as.numeric(format(Real_estate$Date, "%Y"))
Real_estate$Month <- as.numeric(format(Real_estate$Date, "%m"))
Real_estate$Day <- as.numeric(format(Real_estate$Date, "%d"))
Real_estate$Group <- as.numeric(Real_estate$Group)

# Since the "Type" variable is a categorical and it is not possible to normalize and use for our NN model i have converted to the numeric (dummy 0/1) variable.
Real_estate$Type2 <- ifelse(Real_estate$Type == "new", 1, 0)

```

Visualization of Relationships.

For the graphical representation of the distribution of real estate prices, the ggplot2 package is used to create a histogram. Each bar in the histogram represents a range of prices, and the height of the bar represents the frequency (count) of data points within that price range.

In this visual representation, one can spot that most homes are around \$10,000 - \$200,000

We can visually assess the relationship between Price_in_USD and Size in this scatter plot. The plot can inform decisions such as pricing strategies, property development focus, or identifying potential market niches.

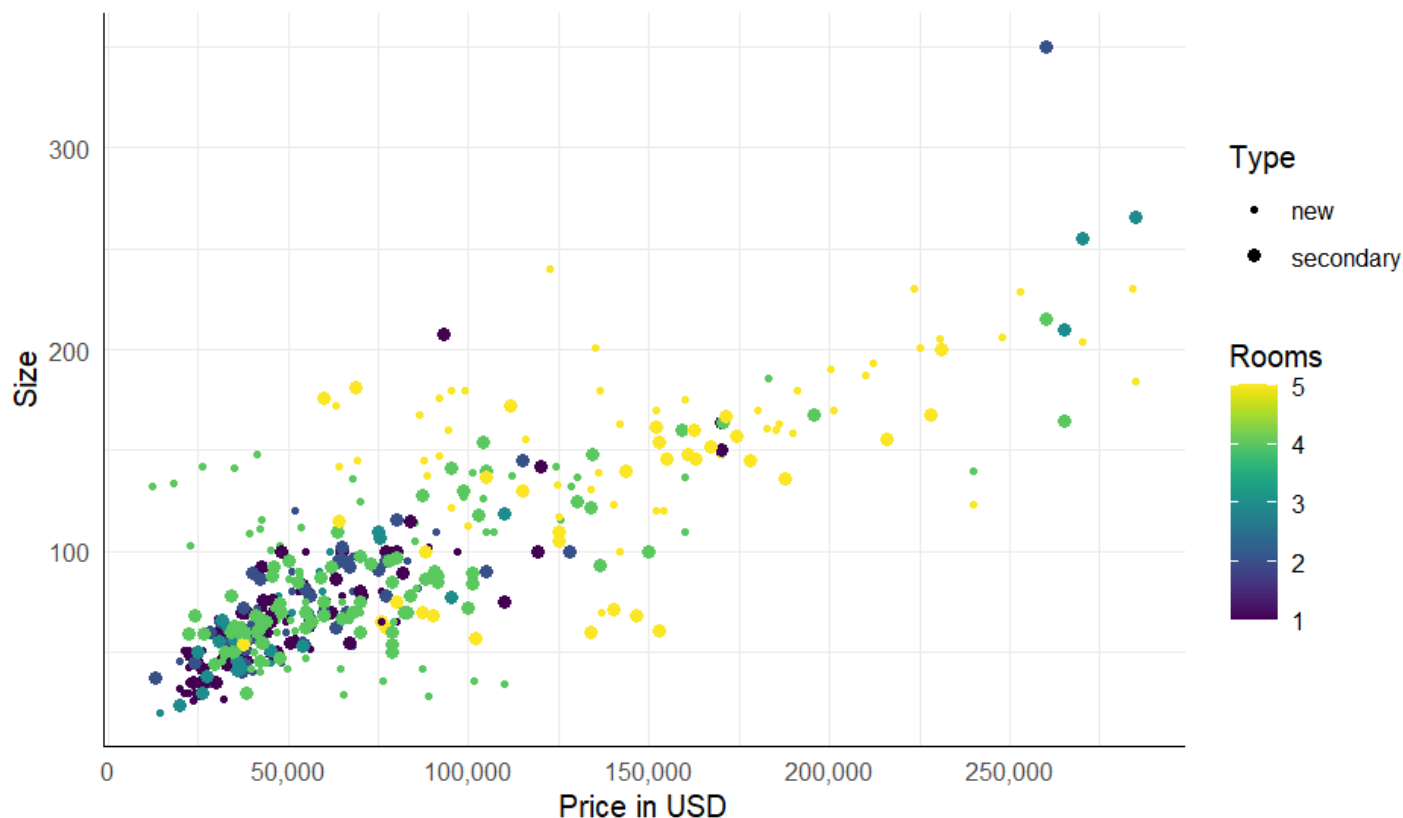
[Hide](#)

```

ggplot(data = Real_estate, aes(x = `Price_in_USD`, y = Size, color = Rooms, size = Type)) +
  geom_point()+
  scale_size_manual(values = c(1,2))+
  labs(x = "Price in USD",
       y = "Size",
       title = "Scatter Plot: Price vs. Size")+
  scale_x_continuous(breaks = seq(0, 285000, by = 50000),
                    #scale_x_continuous is used to customize the breaks and labels on the x-axis.
                    labels = scales::comma_format()+
  theme_minimal()+
  theme(axis.line = element_line(color = "#000000"))+
  scale_color_viridis()

```

Scatter Plot: Price vs. Size



Modelling.

Model 1.1 Linear Regression using LM.

“Price_in_USD” This is the dependent variable. It represents the real estate prices in USD that I try to predict.

“Area”, “Size”, “Rooms”, “Latitude”, “Longitude”, “Type2”, “Group”, “Day”, “Year”, and “Month” are used as independent variables that I use to predict the real estate prices.

[Hide](#)

```
set.seed(123)
model_lm <- lm(data = Real_estate, Price_in_USD ~ Area + Size + Rooms + Latitude + Longitude + Type2 + Group + Day + Year + Month)

predicted_values <- predict(model_lm)
residuals_lm <- Real_estate$Price_in_USD - predicted_values

MSE_lm <- mean(residuals_lm^2)
R_squared <- 1 - (sum(lm(data = Real_estate, Price_in_USD ~ Area + Size + Rooms + Latitude + Longitude + Type2 + Group + Day + Year + Month)$residuals^2) / sum(lm(data = Real_estate, Price_in_USD ~ 1)$residuals^2))

print(paste("Mean Squared Error (MSE):", MSE_lm))
```

```
[1] "Mean Squared Error (MSE): 984815722.01334"
```

[Hide](#)

```
print(paste("R-squared:", R_squared))
```

```
[1] "R-squared: 0.683218951653478"
```

Model 1.2 lm with PCA.

Original Model: R-squared: 0.6832 (By using all 10 original features directly.) #PCA Model: R-squared: 0.6386 (By using 7 principal components derived from the original features, capturing 90.2% of the variance.) #Employing Principle componets did not provide significant benefit to the 'lm' Model Performance: The PCA model's R-squared is slightly lower than the original model's, indicating a minor loss in explanatory power. However, this loss might be acceptable given the benefits of dimensionality reduction, such as reduced risk of overfitting and potentially improved model generalization. It shows that PCA can effectively reduce the dimensionality of my dataset while maintaining almost similar model performance.

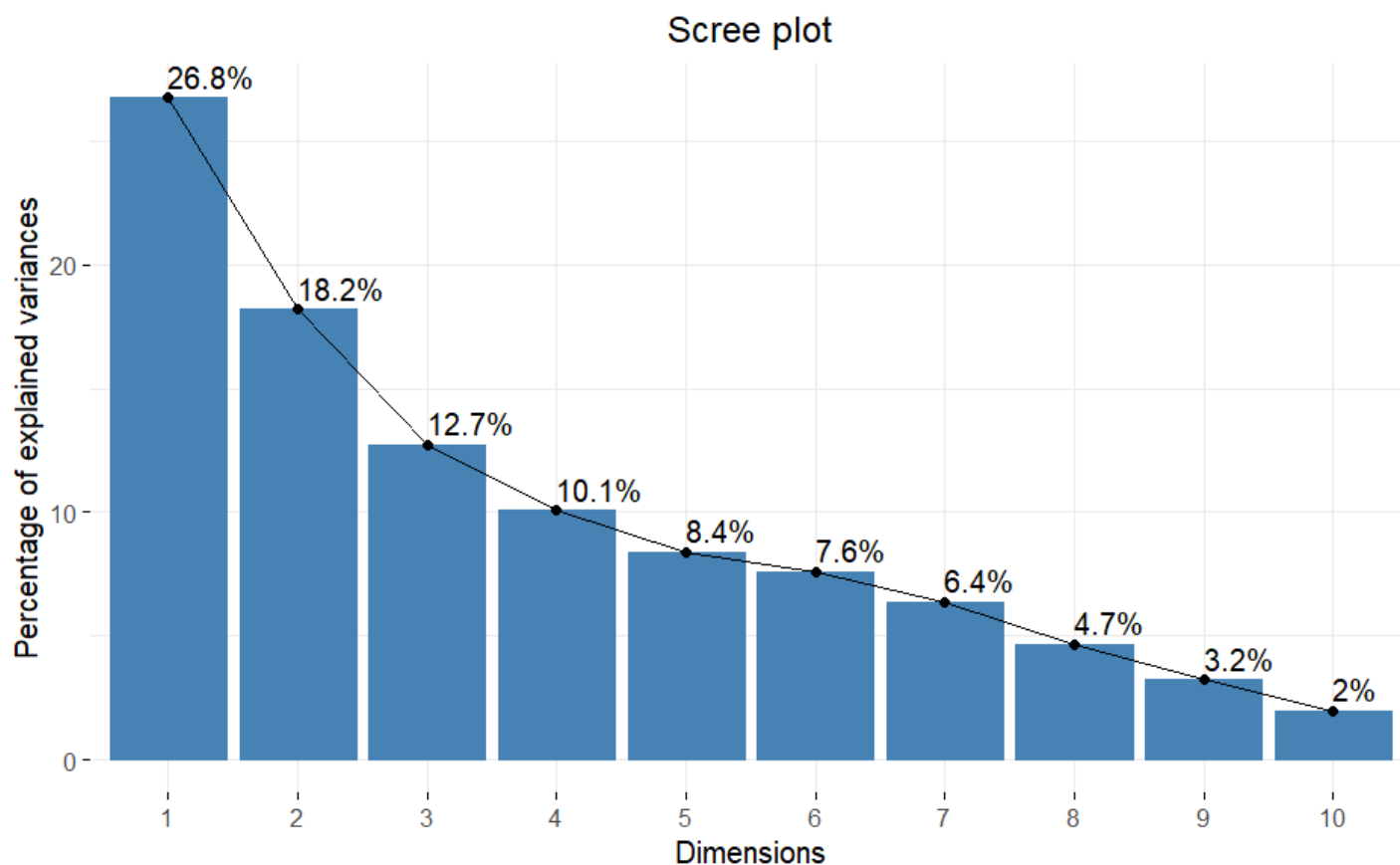
Hide

```
library(factoextra)

# Prepared the data for PCA
dt_pca_lm <- data.table(scale(Real_estate[,c("Area", "Size", "Rooms", "Latitude", "Longitude",
                                             "Type2", "Group", "Day", "Year", "Month")]))

# Performed PCA
model_pca <- prcomp(dt_pca_lm)

# Plot of the explained variance. Contribution of 8 principle components. 6 principal components explain a significant po
rtion of the variance (i.e. 83.8%).
fviz_eig(model_pca, addlabels = TRUE) +
  theme(plot.title = element_text(hjust = 0.5))
```



Hide

```
# Chose the number of components manually, in this case, 7 PCs, since they cover 90.2% of the variance.
num_components <- 7

# Create a new dataset with the selected principal components
new_pca_data_lm <- as.data.frame(model_pca$x[, 1:num_components])
new_pca_data_lm$Price_in_USD <- Real_estate$Price_in_USD

# Fit a regression model using the principal components
model_pca_lm <- lm(Price_in_USD ~ ., data = new_pca_data_lm)

predicted_values_lm_with_pca <- predict(model_pca_lm)
residuals_lm_pca <- Real_estate$Price_in_USD - predicted_values_lm_with_pca

MSE_lm_pca <- mean(residuals_lm_pca^2)
print(paste("Mean Squared Error without PCA (MSE):", MSE_lm))
```

```
[1] "Mean Squared Error without PCA (MSE): 984815722.01334"
```

Hide

```
print(paste("Mean Squared Error with PCA (MSE):", MSE_lm_pca))
```

```
[1] "Mean Squared Error with PCA (MSE): 1123424673.80053"
```

Hide

```
#Summary
original_r_squared <- summary(model_lm)$r.squared
pca_r_squared <- summary(model_pca_lm)$r.squared

print(paste("Original R-squared:", original_r_squared))
```

```
[1] "Original R-squared: 0.683218951653478"
```

Hide

```
print(paste("R-squared with PCA:", pca_r_squared))
```

```
[1] "R-squared with PCA: 0.638633261076154"
```

Model 2.1 KNN-Regression.

Prediction of the 'Price_in_USD' variable based on the 'Size', 'Rooms', 'Area', 'Latitude', 'Longitude', 'Type2', 'Group', 'Day', 'Year', and 'Month'

First I splitted the data into training_data. (By splitting the dataset, I can train my model on one portion (training set) and evaluate its performance on another, unseen portion (testing set). Decided to take into consideration 10 nearest neighbors, since the line with k=10 becomes more flexible. (After having examined the charts of the prediction line changes with different values of k.)

Hide

```
data <- Real_estate[, c("Size", "Rooms", "Area", "Latitude", "Longitude", "Type2", "Group", "Day", "Year", "Month")]
target_variable <- Real_estate$Price_in_USD

model_knn <- knn.reg(train = data, y=target_variable, k=10)

predicted_values_knn <- model_knn$pred #predict(model_knn)
residuals_knn <- target_variable - predicted_values_knn

MSE_knn <- mean(residuals_knn^2)

print(paste("Mean Squared Error of model_knn:", MSE_knn))
```

```
[1] "Mean Squared Error of model_knn: 1105202661.28318"
```

Hide

```
print(model_knn)
```

```
PRESS = 5.07288e+11
R2-Predict = 0.6444947
```

K-Nearest Neighbors. To make a prediction for a new data point, the

algorithm identifies the K nearest neighbors of that point in the feature space. For regression, the predicted output for the new data point is often the average or weighted average of the output values of its K nearest neighbors.

By setting k to different values (1,5,10,20), we can investigate the effect of the smoothness of our regression function.

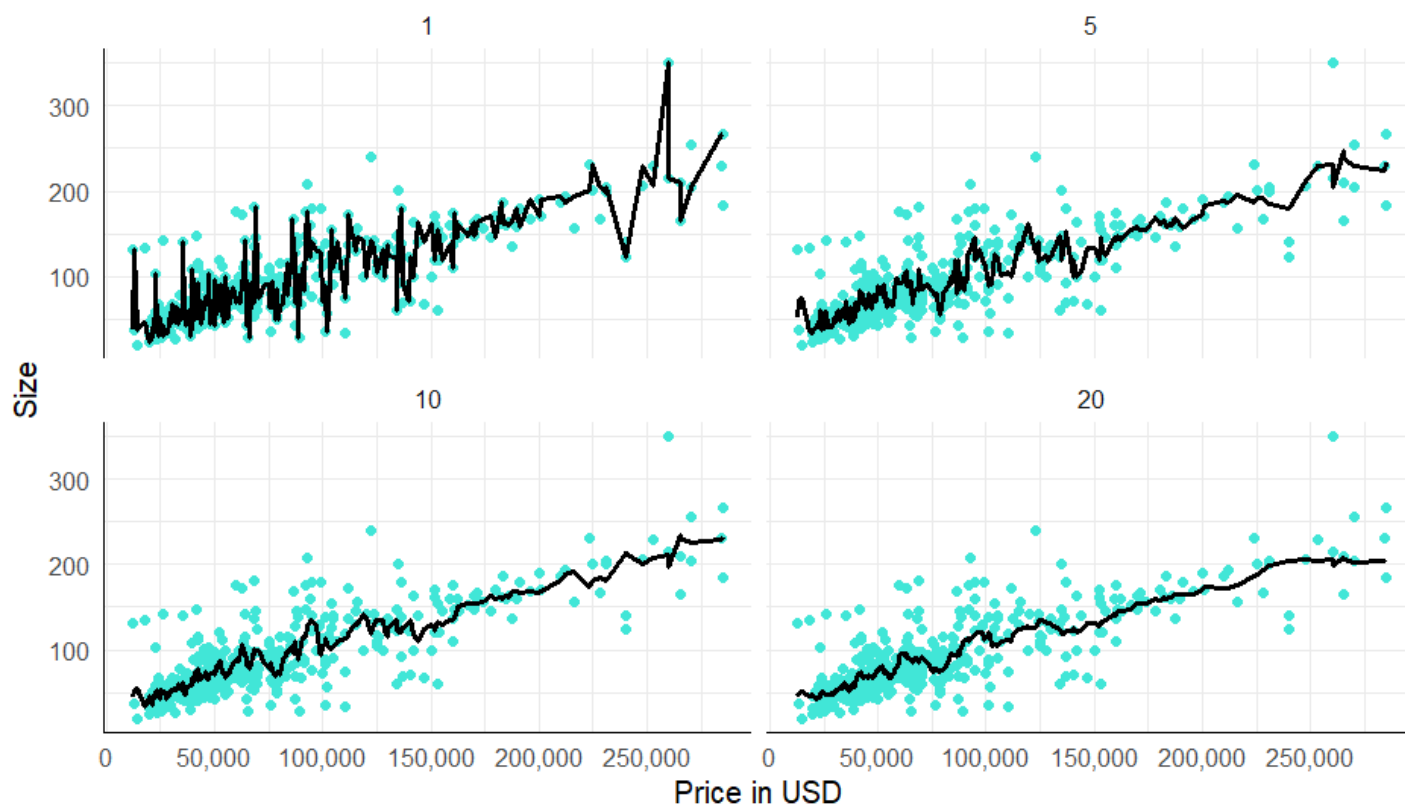
Hide

```
knn <- list()
k <- c(1,5,10,20)
for (i in 1:length(k)) {
  knn[[i]] <- data.table(y_pred = knn.reg(train = Real_estate$Price_in_USD, y=Real_estate$Size, k=k[i])$pred,
                        x=Real_estate$Price_in_USD, k=k[i])
}

dt_knn <- rbindlist(knn)

ggplot(data = dt_knn, aes(x=x,y=y_pred)) +
  geom_point(data = Real_estate, aes(x=Price_in_USD, y=Size), color = "#43e8d8") +
  geom_line(color="black", linewidth=1) +
  facet_wrap(k ~ .) +
  labs(x = "Price in USD",
       y = "Size",
       title = "Price vs. Size with KNN Prediction Lines with different values of k.") +
  scale_x_continuous(breaks = seq(0, 285000, by = 50000),
                    labels = scales::comma_format()) +
  theme_minimal() +
  theme(axis.line = element_line(color = "#000000"))
```

Price vs. Size with KNN Prediction Lines with different values of k.



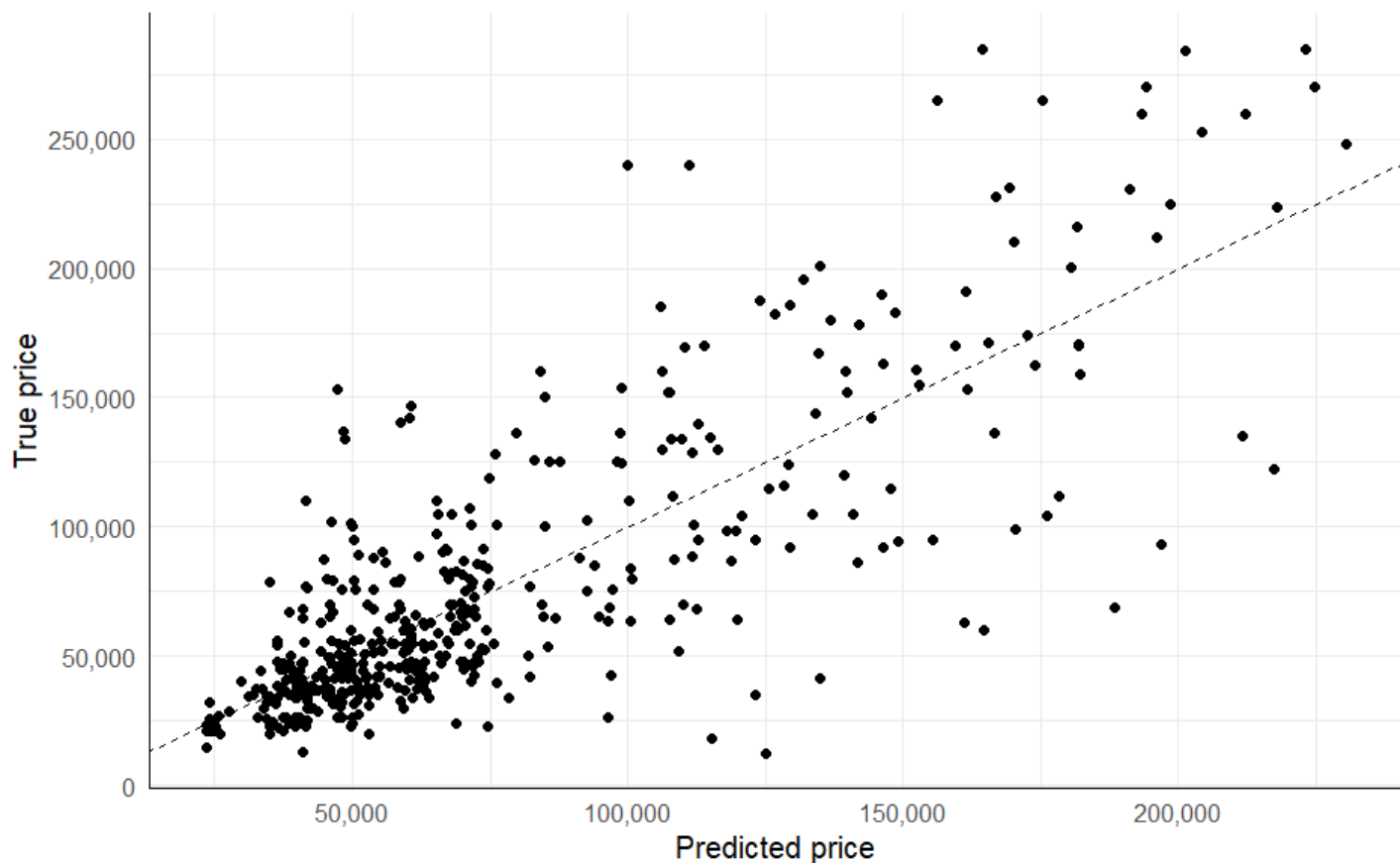
The plot that shows the predicted values against the true values.

Plot of points where the x-axis represents the predicted values from the KNN model (model_knn pred), and the y-axis represents the true values (Real_estate\$Price_in_USD).

The purpose of including this reference line is to provide a visual guide for assessing the quality of the model predictions. When the points in the scatter plot are close/cluster (not deviate from the line) to this reference line, it suggests that the predicted values are similar to the true values. The closer the points are to the line, the better the model's performance. (serves as a benchmark for comparison.) The goal is to have a line that is easy to see and compare against the distribution of points.

[Hide](#)

```
ggplot()+
  geom_point(aes(x=model_knn$pred, y=Real_estate$Price_in_USD)) +
  geom_abline(slope = 1, intercept = 0, linetype="dashed") +
  labs(
    x = "Predicted price",
    y = "True price") +
  scale_x_continuous(breaks = seq(0, 285000, by = 50000),
    #scale_x_continuous is used to customize the breaks and labels on the x-axis.
    labels = scales::comma_format()+
  scale_y_continuous(breaks = seq(0, 285000, by = 50000),
    #scale_x_continuous is used to customize the breaks and labels on the x-axis.
    labels = scales::comma_format()+
  theme_minimal() +
  theme(axis.line = element_line(color = "#000000"))
```



This Feature Engineering part from the first Coursework.

After having done the Engineering part, I have come to realize that, the MSE for the new “lm” model has decreased to 950163375 from 984815722, (by 34652347).

[Hide](#)

```
Real_estate[, Size_sqr := Size^2]
Real_estate[, Size_log := log(Size)]
model_lm_sqr_log <- lm(data = Real_estate, Price_in_USD ~ Area + Size + Rooms + Latitude + Longitude + Type2 + Group + Day + Year + Month + Size_sqr + Size_log)
#summary(model_lm_sqr_log)

mean(model_lm_sqr_log$residuals^2)
```

```
[1] 950163375
```

Engineering part of the second model (model_knn).

Let us have a look at new indicators of accuracy such as “PRESS” and “R2-Predict”, since they indicate an accuracy and it is calculated by summing the squared differences between predicted and observed values.

After squaring the variable “Size” and adding the variable of “inverse of Area”, we get less “prediction sum of Squares” and the R2-Predict closer to 1, which means a good fit of the model to new data. PRESS = 507288000000. > 493607540832 (new lower outcome which is desirable) R2-Predict = 0.6444947 < 0.6540819 (new greater, closer to 1, outcome which is desirable)

[Hide](#)

```
Real_estate[, Area_r := 1/Area]
Real_estate[, Size_log := Size^2]

train_data_2 <- Real_estate[, c("Size", "Rooms", "Area", "Latitude", "Longitude", "Type2", "Group", "Day", "Year", "Month", "Area_r", "Size_log")]
target_variable_2 <- Real_estate$Price_in_USD

model_knn_2 <- knn.reg(train = train_data_2, y=target_variable_2, k=10)

print(model_knn_2)
```

```
PRESS = 493607540832
R2-Predict = 0.6540819
```

Model 2.2. K-nn model with PCA.

Original R-squared: 0.6444 (By using all 10 original features directly.) R-squared with PCA: 0.5888 (By using 5 principal components derived from the original features, capturing 83.8% of the variance.) After reducing the dimensionality, the PCA model's performance is slightly lower than the original model's, but the good news is by applying PCA of my dataset I reduced the dimensionality from 10D to the half 5D reducing the risk of overfitting!

Hide

```
library(class)
# Prepared the data for PCA
dt_pca_knn <- data.table(scale(Real_estate[,c("Size", "Rooms", "Area", "Latitude", "Longitude", "Type2", "Group", "Day", "Year", "Month")]))

# Performed PCA
model_pca_knn <- prcomp(dt_pca_knn)

num_components <- 5

# A new dataset with the selected principal components
new_pca_data_knn <- as.data.frame(model_pca$x[, 1:num_components])
new_pca_data_knn$Price_in_USD <- target_variable

# Fit a regression model using the principal components
model_pca_knn <- knn.reg(train = new_pca_data_knn[, 1:num_components],
                        y = new_pca_data_knn$Price_in_USD,
                        k = 10)

y_actual <- target_variable
y_pred_pca <- model_pca_knn$pred # Predictions
pca_r_squared <- 1 - sum((y_actual - y_pred_pca)^2) / sum((y_actual - mean(y_actual))^2)

predicted_values_knn_pca <- model_pca_knn$pred #predict(model_knn)
residuals_knn_pca <- target_variable - predicted_values_knn_pca

MSE_knn_pca <- mean(residuals_knn_pca^2)

print(paste("Mean Squared Error of model_knn:", MSE_knn_pca))
```

```
[1] "Mean Squared Error of model_knn: 1278188599.28497"
```

Hide

```
print(paste("PCA R-squared:", pca_r_squared))
```

```
[1] "PCA R-squared: 0.588851075977648"
```

Model 3.1. Boosting.

Hide

```
library(gbm)

Boo_data <- Real_estate[, c("Size", "Rooms", "Area", "Latitude", "Longitude", "Type2", "Group", "Day", "Year", "Month")]
Boo_target_variable <- Real_estate$Price_in_USD

# Splitted the data into training and testing sets.
set.seed(123)
train_index <- createDataPartition(Boo_target_variable, p = 0.8, list = FALSE) #train_index: A vector of row indices, randomly selected and stratified based on the target variable. This createDataPartition function picks the rows randomly, creates a random, stratified (has a similar distribution of these classes or values) 80% sample for training

# Train data.
train_data <- Boo_data[train_index, ] #from the data set Boo_data
train_target <- Boo_target_variable[train_index] #extracts the corresponding Price_in_USD values for these rows derived 1 step back.

# Test data.
test_data <- Boo_data[-train_index, ]
test_target <- Boo_target_variable[-train_index]

# Combined the training data with the target variable for the 'gbm' function
train_data_gbm <- data.frame(train_data, Price_in_USD = train_target)

# Training of the GBM model.
set.seed(123)
model_gbm <- gbm(Price_in_USD ~ ., data = train_data_gbm, distribution = "gaussian", n.trees = 100)

# Predict on the test set.
pred_gbm <- predict(model_gbm, newdata = test_data, n.trees = 100)

# Evaluation metrics
residuals_gbm <- Boo_target_variable - pred_gbm
```

```
Warning in Boo_target_variable - pred_gbm :
  longer object length is not a multiple of shorter object length
```

Hide

```
MSE_gbm <- mean(residuals_gbm^2)
gbm_r_squared <- cor(test_target, pred_gbm)^2

print(paste("Mean Squared Error of model_gbm:", MSE_gbm))
```

```
[1] "Mean Squared Error of model_gbm: 4924597662.61715"
```

Hide

```
print(paste("Gradient Boosting R-squared:", gbm_r_squared))
```

```
[1] "Gradient Boosting R-squared: 0.770888290785441"
```

Hide

```
#Results of Boosting
Boo_results <- data.frame(
  Actual = test_target,
  Predicted = pred_gbm
)
head(Boo_results)
```

Model 3.2. Gradient Boosting Machine (GBM) with PCA.

To decide the number of principal components I used "which(cumulative_variance >= 0.9)" which finds the minimum number of components that together explain at least 90% of the variance in the data. For the comparison of the performance of the model with PCA we can take a look at R-squared again which shows the value equal to 0.63, while model gbm itself was showing the value equal to 0.77.

Hide

```
scaled_data_gbm <- scale(Boo_data)
gbm_pca_result <- prcomp(scaled_data_gbm)

#Number of Components
cumulative_variance <- cumsum(gbm_pca_result$sdev^2 / sum(gbm_pca_result$sdev^2)) #pca_result$sdev: Standard deviations of the principal components.
num_components <- which(cumulative_variance >= 0.9)[1] #which(cumulative_variance >= 0.9)[1]: Finds the minimum number of components that together explain at least 90% of the variance in the data.

# Extract the selected principal components
new_gbm_pca_data <- as.data.frame(gbm_pca_result$x[, 1:num_components])
new_gbm_pca_data$Price_in_USD <- target_variable

# Splitting the data into training and testing sets
set.seed(123)
train_index_gbm_pca <- createDataPartition(new_gbm_pca_data$Price_in_USD, p = 0.8, list = FALSE)
train_data_gbm_pca <- new_gbm_pca_data[train_index_gbm_pca, ]
test_data_gbm_pca <- new_gbm_pca_data[-train_index_gbm_pca, ]

# Training GBM model using the principal components
set.seed(123)
model_gbm_pca <- gbm(Price_in_USD ~ ., data = train_data_gbm_pca,
                     distribution = "gaussian",
                     n.trees = 100,
                     shrinkage = 0.1)

# Predictions on the testing data
pred_gbm_pca <- predict(model_gbm_pca, newdata = test_data_gbm_pca, n.trees = 100)

# The same Evaluation metrics
residuals_gbm_pca <- Boo_target_variable - pred_gbm_pca
```

```
Warning in Boo_target_variable - pred_gbm_pca :
  longer object length is not a multiple of shorter object length
```

Hide

```
MSE_gbm_pca <- mean(residuals_gbm_pca^2)
gbm_pca_r_squared <- cor(test_data_gbm_pca$Price_in_USD, pred_gbm_pca)^2

print(paste("Mean Squared Error of model_gbm with PCA:", MSE_gbm_pca))
```

```
[1] "Mean Squared Error of model_gbm with PCA: 4306120700.45061"
```

Hide

```
print(paste("R-squared for GBM model with PCA:", gbm_pca_r_squared))
```

```
[1] "R-squared for GBM model with PCA: 0.635699366383337"
```

Investigating Hyperparameters of GBM model:

Using grid search to investigate the hyperparameters of a GBM model.

Analyzing the output: From the snippet below I took the combination that gives me the best performance metrics i.e. Optimal Hyperparameters (e.g., lowest RMSE or highest R-squared). In this case, it seems the row with shrinkage = 0.100, interaction.depth = 1, n.minobsinnode = 20, n.trees = 100 has the highest R-squared (0.5788756) and relatively low RMSE (36771.60) and relatively low RMSESD (3371.38).

Hide

```
gbm_grid = expand.grid(interaction.depth = 1:5,
                        n.trees = c(100,300,500,1000),
                        shrinkage = c(0.1, 0.01, 0.001),
                        n.minobsinnode = c(10,20))

train_settings <- trainControl(method = "cv", number = 10)

caret_gbm <- train(Price_in_USD ~ ., data = new_gbm_pca_data,
                  method = "gbm",
                  trControl = train_settings,
                  tuneGrid = gbm_grid,
                  verbose = FALSE)

#caret_gbm
head(caret_gbm$results)
```

Hide

NA

(Model Neural Network didn't come off, despite my rigorous and hourly efforts. :(Sorry for that!)

Model 4.1. Random Forest.

Hide

```
library(randomForest)
Bag_data <- Real_estate[, -c("Tashkent_city_districts", "Type", "Date", "residuals",
                             "Size_sqr", "Size_log")]
```

```
Warning in `[.data.table`(Real_estate, , -c("Tashkent_city_districts", "Type", :
  column(s) not removed because not found: [residuals]
```

Hide

```
Bag_target_variable <- Real_estate$Price_in_USD

training_sample <- Bag_data[, sample(.N, floor(.N*.80))]
Bag_training <- Bag_data[training_sample]
Bag_test <- Bag_data[-training_sample]

RF_model <- randomForest(Price_in_USD ~ ., data=Bag_training, ntree=500)

# Predictions on the test set
predictions_RF <- predict(RF_model, newdata = Bag_test)

# Evaluation of the model
actual <- Bag_test$Price_in_USD
MSE_RF <- mean((predictions_RF - actual)^2)
cat("Mean Squared Error of RF model:", MSE_RF, "\n")
```

```
Mean Squared Error of RF model: 933528200
```

Hide

```
r_squared_RF <- cor(predictions_RF, actual)^2
cat("R-squared of RF model:", r_squared_RF, "\n")
```

```
R-squared of RF model: 0.716767
```

Model 4.2. Random Forest with PCA.

Hide

```
scaled_RF_data <- scale(Bag_data)
model_RF_pca <- prcomp(scaled_RF_data)

num_components4 <- 6

new_RF_pca_data <- as.data.frame(model_pca$x[, 1:num_components4])
new_RF_pca_data$Price_in_USD <- Bag_target_variable

# Split the data into training and testing sets
set.seed(123)
train_index_RF_pca <- createDataPartition(new_RF_pca_data$Price_in_USD, p = 0.8, list = FALSE)
train_data_RF_pca <- new_RF_pca_data[train_index_RF_pca, ]
test_data_RF_pca <- new_RF_pca_data[-train_index_RF_pca, ]

original_price <- new_RF_pca_data$Price_in_USD

RF_PCA_model <- randomForest(Price_in_USD ~ ., data=train_data_RF_pca, ntree=500)

# Predictions on the testing data
pred_RF_pca <- predict(RF_PCA_model, newdata = test_data_RF_pca, n.trees = 500)

# Evaluation of the performance of the new with PCA model (R-squared)
RF_pca_r_squared <- cor(test_data_RF_pca$Price_in_USD, pred_RF_pca)^2
mse <- mean((pred_RF_pca - original_price)^2)
```

```
Warning in pred_RF_pca - original_price :
  longer object length is not a multiple of shorter object length
```

Hide

```
rmse <- sqrt(mse)

cat("Mean Squared Error of RF model with PCA:", mse, "\n")
```

```
Mean Squared Error of RF model with PCA: 4358230445
```

Hide

```
cat("R-squared RF model with PCA:", RF_pca_r_squared, "\n")
```

```
R-squared RF model with PCA: 0.6684627
```

Hide

```
cat("Root Mean Squared Error:", rmse, "\n")
```

```
Root Mean Squared Error: 66016.9
```

Investigating Hyperparameters of Random Forest model:

The optimal value for mtry is 7, which gives the best model performance with the lowest RMSE (30874.24) and highest R-squared (0.7180725).

Hide

```
library(data.table)
library(caret)
rf_grid <- data.table("mtry" = 1:(ncol(Bag_training)-1))

train_settings <- trainControl(method = "cv", number = 5)

caret_rf <- train(Price_in_USD ~ ., data = Bag_training,
  method = "rf",
  ntrees = 500,
  trControl = train_settings,
  search = "grid",
  tuneGrid = rf_grid)

print(caret_rf)
```

Random Forest

367 samples
11 predictor

No pre-processing

Resampling: Cross-Validated (5 fold)

Summary of sample sizes: 294, 293, 294, 294, 293

Resampling results across tuning parameters:

mtry	RMSE	Rsquared	MAE
1	39995.04	0.5743852	28745.33
2	34855.75	0.6306174	23899.09
3	33412.75	0.6491468	22511.90
4	32731.33	0.6599254	21815.36
5	32683.32	0.6597356	21546.55
6	32636.27	0.6595886	21394.43
7	32551.17	0.6617234	21260.58
8	32564.61	0.6632814	21170.05
9	32584.49	0.6619128	21201.13
10	32666.67	0.6621618	21232.11
11	32727.44	0.6631534	21201.46

RMSE was used to select the optimal model using the smallest value.

The final value used for the model was mtry = 7.

Generalization

Training best 4 Models with k-fold Cross-Validation.

Models to evaluate:

- Model 1.1 Linear Regression
- Model 2.1 K Nearest Neighbor
- Model 3.2 Gradient Boosting Machine with PCA
- Model 4.1 Random Forest

These models perform relatively better than the others when comparing their MSE.

Hide

```
# Model 1.1 lm
#model_lm <- lm(data = Real_estate, Price_in_USD ~ Area + Size + Rooms + Latitude + Longitude + Type2 + Group + Day + Year + Month)
set.seed(123)
train_control_1.1 <- trainControl(method = "cv", number = 10)

model_cv_lm <- train(
  Price_in_USD ~ Area + Size + Rooms + Latitude + Longitude + Type2 + Group + Day + Year + Month,
  data = Real_estate,
  method = "lm",
  trControl = train_control_1.1)

#print(model_cv_lm)
#This extracts the RMSE for each fold and calculates the average MSE.
resampling_results_lm <- model_cv_lm$resample
MSE_cv_lm <- mean(resampling_results_lm$RMSE^2)

print(paste("MSE of cross-validated model_lm:", MSE_cv_lm))
```

```
[1] "MSE of cross-validated model_lm: 1019654442.50161"
```

The slight increase in the cross-validated MSE of the model_lm compared to the original MSE is expected because cross-validation generally provides a more realistic estimate of the model's performance on unseen data.

[Hide](#)

```
# Model 2.1 knn

#data <- Real_estate[, c("Size", "Rooms", "Area", "Latitude", "Longitude", "Type2", "Group", "Day", "Year", "Month")]
#target_variable <- Real_estate$Price_in_USD
#model_knn <- knn.reg(train = data, y=target_variable, k=10)

train_control_2.1 <- trainControl(method = "cv", number = 20)

model_cv_knn <- train(
  Price_in_USD ~ Area + Size + Rooms + Latitude + Longitude + Type2 + Group + Day + Year + Month,
  data = Real_estate,
  method = "knn",
  trControl = train_control_2.1)

#print(model_cv_lm)
#This extracts the RMSE for each fold and calculates the average MSE.
resampling_results_knn <- model_cv_knn$resample
MSE_cv_knn <- mean(resampling_results_knn$RMSE^2)

print(paste("MSE of cross-validated model_knn:", MSE_cv_knn))
```

```
[1] "MSE of cross-validated model_knn: 1128769397.99249"
```

The MSE values before and after cross-validating the KNN model being very close, indicating that the KNN model is quite stable and is performing consistently both on the whole dataset and during cross-validation. This is a good sign as it suggests that the model generalizes well to unseen data.

[Hide](#)

```
# Model 3.2 gbm

train_control_3.2 <- trainControl(method = "cv", number = 20)

model_cv_gbm <- train(
  Price_in_USD ~ Area + Size + Rooms + Latitude + Longitude + Type2 + Group + Day + Year + Month,
  data = Real_estate,
  method = "gbm",
  trControl = train_control_3.2,
  verbose = FALSE
)

#print(model_cv_lm)
#This extracts the RMSE for each fold and calculates the average MSE.
resampling_results_gbm <- model_cv_gbm$resample
MSE_cv_gbm <- mean(resampling_results_gbm$RMSE^2)

print(paste("MSE of cross-validated model_gbm:", MSE_cv_gbm))
```

```
[1] "MSE of cross-validated model_gbm: 915385341.792045"
```

The primary goal of using cross-validation for the Gradient Boosting Machine (GBM) with PCA model is achieved. The substantial decrease in MSE from before CV to after CV indicates that cross-validation has significantly improved the model's performance. This improvement suggests that the model is better tuned and generalized to new data.

[Hide](#)

```
# Model 4.1 RF

train_control_4.1 <- trainControl(method = "cv", number = 20)

model_cv_RF <- train(
  Price_in_USD ~ Area + Size + Rooms + Latitude + Longitude + Type2 + Group + Day + Year + Month + Area_r,
  data = Real_estate,
  method = "rf",
  trControl = train_control_4.1)

print(model_cv_RF)
```

Random Forest

459 samples
11 predictor

No pre-processing

Resampling: Cross-Validated (20 fold)

Summary of sample sizes: 436, 435, 437, 435, 437, 435, ...

Resampling results across tuning parameters:

mtry	RMSE	Rsquared	MAE
2	32711.09	0.6843069	23410.05
6	30426.32	0.6967195	20474.79
11	30638.95	0.6948756	20478.31

RMSE was used to select the optimal model using the smallest value.
The final value used for the model was mtry = 6.

[Hide](#)

```
#This extracts the RMSE for each fold and calculates the average MSE.  
resampling_results_RF <- model_cv_RF$resample  
MSE_cv_RF <- mean(resampling_results_RF$RMSE^2)  
  
print(paste("MSE of cross-validated model_RF:", MSE_cv_RF))
```

```
[1] "MSE of cross-validated model_RF: 967470508.032645"
```

The model with mtry = 10 is selected as the optimal model based on the smallest RMSE.

Analysis:

Based on the Mean Squared Error (MSE) values derived from each model after cross-validation, we can determine the optimal model by selecting the one with the lowest MSE. Here are the MSE values for each model:

1. Linear Model (model_lm): 1,019,654,442.50161
2. k-Nearest Neighbors (model_knn): 1,128,769,397.99249
3. Gradient Boosting Machine with PCA (model_gbm): 4,306,120,700.45061
4. Random Forest (model_RF): 956,082,864.094494

"MSE of cross-validated model_lm: 1019654442.50161" "MSE of cross-validated model_knn: 1128769397.99249" "MSE of cross-validated model_gbm: 913703070.65313" "MSE of cross-validated model_RF: 956082864.094494"

The Gradient Boosting Machine (GBM) model has the lowest MSE at 913703070.65313, indicating it has the best performance among the models tested. If I have to select the Optimal Model: The GBM model with PCA would be selected as the optimal model based on the lowest cross-validated MSE. 'model_cv_gbm_pca'